实验三：MPI 分布式图神经网络推理

本实验从单机多线程走向多机多进程。需要实现两种 MPI 通信策略，并与一个朴素的全量广播基线作对比。

单机多线程时，所有线程共享同一份数据，通信由硬件的缓存一致性隐式完成。换成多台机器后，进程之间没有共享内存，数据要显式地传过去。图计算在这里格外棘手：图一旦切开，节点的邻居就可能落在别的机器上。本次实验要比较的，就是两种把这些边界数据传过来的方式。

实验在华为云鲲鹏 ECS 集群上完成。集群的购买与配置见附录 A，建议先按附录 A 把 4 台机器和免密、MPI 装好，再回到第 1 节。

1 计算过程

1.1 图卷积

图有 `N` 个节点，每个节点有 `d` 维特征，邻接关系用 CSR（Compressed Sparse Row）存储。一层图卷积为

$$H^{(l+1)} = \sigma\Big(\big(D^{-1}AH^{(l)}\big)\,W^{(l)}\Big), \qquad \big(D^{-1}AH\big)[v] = \frac{1}{\deg(v)}\sum_{u\in N(v)} H[u].$$

也就是：先把每个节点的邻居特征取平均（邻居聚合），再做一次线性变换，最后过 ReLU。本实验最后一层不加激活。

`D^-1 A H` 是稀疏矩阵乘稠密矩阵（SpMM），相当于把稀疏矩阵向量乘（SpMV）中的向量换成了一个 `N × d` 的矩阵。每个非零元素带来的算术操作很少，所以性能主要受访存和负载均衡影响；放到多机环境后，还要再加上一项通信开销。

本实验只做推理（前向传播），不做训练。权重由固定种子生成，所有进程得到相同的权重，因此权重不需要通信。

1.2 划分与 halo

每个进程负责一段节点，保存它们的特征。计算第 `l+1` 层时，进程需要所有邻居在第 `l` 层的表示，而其中一部分邻居属于别的进程。这些"本进程计算时需要、但不属于本进程"的节点叫 halo 节点。

第 `l+1` 层用的是邻居在第 `l` 层的表示，而这个表示每层都在变，所以 `L` 层网络要交换 `L` 次。

配套代码使用统一的本地编号空间：`[0, localN)` 是本进程拥有的节点，`[localN, localN + haloN)` 是 halo 区。

1.3 通信量

设进程 `p` 的 halo 节点数为 `h_p`，第 `l` 层维度为 `d_l`。一次完整推理中，进程 `p` 接收的数据量约为

$$\sum_{l=0}^{L-1} h_p \cdot d_l \cdot \texttt{sizeof(float)}.$$

注意 `h_p` 由图的划分质量决定，而不是由进程数简单决定；多条边指向同一个远程节点时，该节点只需要传一次。所以跨进程边数和 halo 节点数是两个不同的量，它们的增长也不成比例。

2 已有代码

编译和运行方法见 `code/README.md`。配套代码可以直接编译运行，但四个函数都是空的，会输出全零并被结果检查判为失败。

已经提供：

- 图加载、按 block / cyclic 两种方式划分、局部 CSR 构建；
- `remapLocalIndices`：给定 halo 节点列表，完成下标重映射；
- 计算核 `spmmLayer` / `spmmLayerRange`（标量实现，在 double 上按固定顺序累加）；
- 串行参考实现 `gnnSerial` 与结果检查 `checkResult`；
- 预热 + 重复计时、中位数/最小/最大值统计；
- `--only` 选项，便于用 perf 做单策略测量；
- halo 规模与跨进程边数的统计。

分阶段计时、各进程负载等实验需要的其他统计量，自行添加。

3 要完成的实现

`main.cpp` 里有四个空函数，签名已固定，函数内部的实现方式不作限定。通信计划的数据结构和存放位置也不作限定，框架代码不会访问它。

`gnnSetup` —— 建立通信计划，在划分之后、计时开始之前调用一次。难点在于每个进程只知道自己需要哪些远程节点，却不知道别人需要自己的哪些节点，而发送方必须知道该发什么。

`gnn_mpi_student_1` —— 用集合通信完成每层的 halo 交换。每个进程发给不同对端的数据量不同，属于不规整通信。

`gnn_mpi_student_2` —— 用点对点通信完成每层的 halo 交换，并让通信与计算重叠。只换接口而不做重叠，不算完成第二种策略。

`gnn_mpi_allgather` —— 对照基线，每层把所有节点的表示收集到所有进程。正确性最容易保证，但每个进程都要存下全图，内存占用不随进程数下降。

可用的接口：`MPI_Alltoall`、`MPI_Alltoallv`、`MPI_Allgatherv`、`MPI_Isend` / `MPI_Irecv` / `MPI_Waitall`。注意 MPI 的 counts / displs 单位是元素个数。

不要调用 PETSc、Trilinos 等库的分布式 SpMM 接口代替自己的 MPI 实现。

4 结果检查

比较并行实现时，图、权重、层数、隐层维度都要相同。不能通过少算节点、跳过某些层或修改参考结果来缩短时间。

`checkResult` 把各进程的局部结果收集到 rank 0，与串行参考逐元素比较，并检查每个节点恰好被一个进程输出。容差 `atol = rtol = 1e-7`。

需要覆盖的情况：单进程与多进程、block 与 cyclic、单层与多层、进程数不整除节点数、孤立点。

5 性能实验

统一使用 `gen_graph.py` 生成的图。默认配置：`N=200000`、`d=64`、平均度数 20、`--layers 2`、`--hidden 64`。

围绕下面四类变量进行实验：

- **进程数**：含单机多进程与跨机器多进程两种分布，用不同的 hostfile 控制，见附录 A.3；
- **划分方式与图结构**：`--part` 与 `--intra`；
- **计算通信比**：`--layers` 与 `--hidden`；
- **负载均衡**：`--skew` 与 `--hub-span`。

`gen_graph.py` 和 `./main` 的输出里有跨进程边数、halo 节点数、block 负载 : max/mean 等统计，可以直接取用。

6 计时与报告指标

每个配置先预热至少一次，再正式重复 10 次，报告中位数、最小值和最大值。单次测量的波动可以达到中位数的 50%，重复次数不要随意调小。配套代码用 `MPI_Wtime`，每次计时前加 `MPI_Barrier`，取各进程耗时的最大值作为墙钟时间。

一次推理的计时应包含打包、通信、等待和计算；图加载、划分、建通信计划、串行参考、结果检查和打印放在计时之外。建通信计划的时间单独报告，程序会打印成"划分+建计划"一行。

指标定义：

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)}, \qquad E(p) = \frac{S(p)}{p}.$$

`T_serial` 始终使用串行参考实现的时间，全文保持一致。各策略使用相同的编译选项，默认 `-O3 -std=c++17`。

7 使用 Linux 性能工具

至少对串行参考、一个 MPI 策略和全量广播基线各做一次测量，选两个与分析相关的计数器解释结果。`lscpu`、`uname -a`、`perf list` 可以查看环境和可用事件。

注意 `mpirun` 下每个进程是独立的，直接 `perf stat mpirun ...` 会把 `mpirun` 本身和所有进程混在一起，需要自己安排采集范围。`--no-check` 会跳过正确性检查和串行参考，避免它们污染计数，此时没有 `T_serial`，加速比一栏显示 `-`；图加载和建通信计划仍会执行。

若环境不允许使用 perf，可以改用 `/usr/bin/time -v` 等工具，说明测到了哪些信息。

8 提交材料

提交一个压缩包，包含代码、实验报告等。图数据文件不用放进压缩包，但要注明生成命令和参数。

实验报告正文 2–4 页，可用 Markdown 或 LaTeX 导出 PDF。正文应包含：

1. 两种通信策略与全量广播基线的实现思路，重点说明通信计划如何建立、重叠如何实现；
2. 实验环境、参数、比较条件和正确性结果；
3. 实验的数据与分析；
4. 性能工具的数据和解释；

5. 结论、遇到的 bug 或负优化，以及方法的局限和扩展性。

正确性截图和完整数据表可以放附录，但主要结论及其依据要在正文中说明。

本实验没有标准答案，不必过于纠结，目的是探索理解 MPI 的原理即可。过程大于结果，多和其他同学交流探讨。

附录 A 实验环境：华为云鲲鹏 ECS 集群

本实验需要多台机器，这是它对环境的主要要求。本附录的配置工作不计入实验难度，但请完整做完——跨机器运行不起来，第 5 节的进程数实验就做不全。

A.1 购买 ECS

登录 https://www.huaweicloud.com ，选择 产品 → 基础服务 → 弹性云服务器 ECS。

按下表购买 4 台：

计费模式	区域	CPU 架构	规格	镜像	系统盘
按需计费	华北-北京四	鲲鹏计算	kc1.xlarge.2（4 vCPU / 8GB）	openEuler 20.03	40GB

网络配置：默认 VPC、Sys-default 安全组、弹性公网 IP（带宽 10Mbit/s）。高级配置：名称设为 `ecs-hw`，设置密码，云备份选择暂不购买。

规格说明：默认数据集（20 万节点、64 维特征）单进程峰值内存约 500MB，`kc1.large.2`（2 vCPU / 4GB）也够用。但第 5 节要求对比"单机多进程"与"跨机器多进程"，4 vCPU 的实例更便于在一台机器上开 4 个进程，所以推荐 `kc1.xlarge.2`。

> 成本提示：按需计费，4 台 `kc1.xlarge.2` 约 1 元/小时。
> 实验结束后请在控制台删除实例，否则会持续计费。只是暂时不用时，
> 可以选择"关机"，但系统盘仍会计费。

A.2 集群配置

以下 A.2.1~A.2.4 的操作四台机器都要执行。以用户名 `student` 为例，四台主机名为 `ecs-hw-0001` ~ `ecs-hw-0004`。

A.2.1 创建用户

```
# root 下执行
adduser student
passwd student
usermod -aG wheel student
```

A.2.2 配置主机名解析

各主机的私网 IP 可在控制台或用 `ifconfig` 查看。集群内部通信走私网，不要用公网 IP。

```
vim /etc/hosts
```

添加（IP 换成实际分配的）：

```
192.168.0.10  ecs-hw-0001
192.168.0.11  ecs-hw-0002
192.168.0.12  ecs-hw-0003
192.168.0.13  ecs-hw-0004
```

A.2.3 配置免密登录

MPI 启动远程进程依赖 SSH 免密。切换到普通用户后执行：

```
su - student
ssh-keygen -t rsa -b 4096          # 一路回车
ssh-copy-id student@ecs-hw-0001    # 包括自己
ssh-copy-id student@ecs-hw-0002
ssh-copy-id student@ecs-hw-0003
ssh-copy-id student@ecs-hw-0004
```

验证：`ssh ecs-hw-0002 hostname` 应当直接返回主机名，不提示密码。必须手动 ssh 到每台机器一次以接受 host key，否则 mpirun 会失败。

A.2.4 安装依赖与 MPI

```
sudo yum -y install gcc-c++ make python3 python3-pip perf
pip3 install --user -i https://repo.huaweicloud.com/repository/pypi/simple numpy
```

MPI 用源码编译安装（鲲鹏平台）：

```
wget http://www.mpich.org/static/downloads/3.3.2/mpich-3.3.2.tar.gz
tar -zxf mpich-3.3.2.tar.gz
cd mpich-3.3.2
./configure --prefix=/usr/local/mpich
make -j$(nproc) && sudo make install
```

把 MPI 加入 PATH（写进 `~/.bashrc`）：

```
echo 'export PATH=/usr/local/mpich/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=/usr/local/mpich/lib:$LD_LIBRARY_PATH' >> ~/.bashrc
source ~/.bashrc
mpic++ --version && mpirun --version
```

> 四台机器的 MPI 必须是相同版本、相同路径，否则 mpirun 启动远程进程会失败。

A.3 验证集群

在 `ecs-hw-0001` 上建立 hostfile：

```
mkdir -p ~/lab3 && cd ~/lab3
cat > hostfile <<'EOF'
ecs-hw-0001:4
ecs-hw-0002:4
ecs-hw-0003:4
ecs-hw-0004:4
EOF
```

`:4` 表示该机器最多启动 4 个进程。这是 MPICH 的写法，Open MPI 要写成 `ecs-hw-0001 slots=4`，用错语法会报 "illegal character in node name"。进程数超过 slots 总和时会报 "not enough slots"，此时要么调整 hostfile，要么加 `--oversubscribe`。

用一个最小程序验证：

```
cat > hello.c <<'EOF'
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    char name[256];
    int len;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(name, &len);
    printf("rank %d / %d on %s\n", rank, size, name);
    MPI_Finalize();
    return 0;
}
EOF
mpicc hello.c -o hello
mpirun --hostfile hostfile -np 4 ./hello
```

应当看到 4 行输出，且主机名各不相同（说明进程确实分布在 4 台机器上）。若全在同一台，检查 hostfile 和免密配置。

对比两种进程分布，用不同的 hostfile 控制：

```
# 跨机器：4 台各 1 个进程
printf 'ecs-hw-0001:1\necs-hw-0002:1\necs-hw-0003:1\necs-hw-0004:1\n' > hostfile_spread

# 单机：1 台开 4 个进程
printf 'ecs-hw-0001:4\n' > hostfile_single
```

A.4 分发代码与数据

把 `code/` 目录下的全部文件放到 `ecs-hw-0001` 的 `~/lab3/`，编译后分发。可执行文件和图数据必须在所有机器的相同路径下存在：

```
cd ~/lab3
make
python3 gen_graph.py --nodes 200000 --degree 20 --feat-dim 64 --out graph.bin

for h in ecs-hw-0002 ecs-hw-0003 ecs-hw-0004; do
    ssh $h "mkdir -p ~/lab3"
    scp main graph.bin hostfile $h:~/lab3/
done
```

每次重新编译后都要重新分发可执行文件。建议写成一个小脚本：

```
cat > sync.sh <<'EOF'
#!/bin/bash
set -e
make
for h in ecs-hw-0002 ecs-hw-0003 ecs-hw-0004; do
    scp -q main $h:~/lab3/
done
echo "已同步到所有节点"
EOF
chmod +x sync.sh
```

> 图数据只需分发一次（除非更换参数重新生成）。注意四台机器上的 `graph.bin` 必须完全一致：不一致时结果检查会失败，但报出来的是大量数值偏差，很难看出根因在数据而不在代码。建议先用 `md5sum graph.bin` 在各机器上核对。

运行：

```
mpirun --hostfile hostfile -np 4 ./main --graph graph.bin
```

参考资料

- MPI 3.1 标准：`MPI_Alltoallv`、`MPI_Allgatherv`、`MPI_Isend` / `MPI_Irecv` / `MPI_Waitall` 的定义
- Open MPI 文档
- GraphSAGE 论文：Hamilton et al., *Inductive Representation Learning on Large Graphs*, NeurIPS 2017
- DGL 的分布式训练文档（了解工业界如何处理同一问题）：https://docs.dgl.ai/guide/distributed.html
- METIS 图划分工具（了解如何减少跨分区边）：http://glaros.dtc.umn.edu/gkhome/metis/metis/overview